Desarrollo Dirigido por Pruebas (TDD)

Escribe primero la prueba, después el código: calidad desde el inicio

Calidad de Software · Actividad 16

El ciclo Rojo – Verde – Refactorizar

1 ROJO
Escribe una prueba automatizada de una pequeña funcionalidad. Al ejecutarla, **falla** porque el código aún no existe.

→

2 VERDE
Escribe el código mínimo necesario para que la prueba **pase**.

→

3 REFACTORIZAR
Mejora el diseño y elimina duplicación sin cambiar el comportamiento; las pruebas deben seguir en verde.

↻ Se repite para cada incremento de funcionalidad

Pasos del TDD

1. **Identificar el incremento de funcionalidad**
   Elegir una mejora pequeña, que pueda implementarse en pocas líneas de código.
2. **Escribir una prueba automatizada para esa funcionalidad**
   La prueba define el comportamiento esperado antes de programar.
3. **Ejecutar la prueba junto con las demás pruebas existentes**
   La nueva prueba debe fallar (rojo); así se comprueba que la prueba realmente detecta la ausencia de la funcionalidad.
4. **Implementar la funcionalidad**
   Escribir solo el código necesario para satisfacer la prueba.
5. **Volver a ejecutar las pruebas**
   Si alguna falla, corregir el código y repetir hasta que todas pasen (verde).
6. **Refactorizar y pasar al siguiente incremento**
   Limpiar el código, confirmar que todo sigue en verde y reiniciar el ciclo con la siguiente funcionalidad.

Beneficios del TDD

✔ Cobertura del código
Cada segmento de código tiene al menos una prueba asociada, porque se escribió para satisfacerla.

✔ Pruebas de regresión
Se construye una suite que permite detectar si un cambio nuevo rompe algo que ya funcionaba.

✔ Depuración simplificada
Cuando una prueba falla, es claro dónde está el problema: en el código recién escrito.

✔ Documentación del sistema
Las pruebas describen qué debe hacer el código y sirven como documentación viva.

Diseño para la facilidad de prueba

Un código es más fácil de probar con TDD cuando:

- Las clases y funciones son pequeñas y con una sola responsabilidad.
- Existe bajo acoplamiento: las dependencias se inyectan y pueden sustituirse por simulaciones (mocks).
- Se programa contra interfaces claras, con entradas y salidas observables.
- Se evita el estado global y los efectos secundarios ocultos.

TDD y los principios ágiles

Retroalimentación rápida
Las pruebas informan en segundos si el cambio funciona, en línea con las entregas frecuentes del desarrollo ágil.

Aceptar el cambio con confianza
La suite de pruebas protege el sistema al modificar requisitos, y la atención continua a la excelencia técnica mantiene la simplicidad.

Fuentes consultadas:
Tutsplus. (s. f.). *The principles of agile development*. Envato Tuts+. https://code.tutsplus.com/the-principles-of-agile-development--net-25732a
Springer. (2023). Capítulo 23, secciones 23.6 *Test-Driven Development* y 23.7 *Design for Testability*. https://link.springer.com/chapter/10.1007/978-3-031-40336-1_23